



Tecnológico de Monterrey

Integrated Activity 1 Reflection

Omar Ávila Meza - A01743547

Analysis and design of advanced algorithms

Instituto Tecnológico de Monterrey Campus Guadalajara

1. Pattern Search

1.1 Problem Description

The first part of the project asks us to check whether the contents of the malicious code files (mcode1.txt, mcode2.txt and mcode3.txt) appear inside the transmission files (transmission1.txt and transmission2.txt). If the code is found, we also need to report the exact position where it starts. In other words, we are looking for an exact sequence of characters inside a much longer sequence of characters, and we have to do it six times in total (three codes against two transmissions).

Since the transmission files can be very large, we needed a search method that does not waste time re-checking characters it has already looked at.

1.2 Algorithm

The most obvious way to solve this would be to compare the pattern against the text at every possible position, one by one. The problem with that approach is that whenever a comparison fails, we have to go back in the text and start over from the next position, throwing away all the work we just did.

KMP avoids that waste. The key idea is that when a mismatch happens, we already know something useful, which characters of the pattern matched right before the mismatch. If part of the beginning of the pattern also appears at the end of what we just matched, we can slide the pattern forward to that point instead of starting from scratch.

To know how far we can slide, KMP first builds a helper array called LPS (Longest Prefix Suffix). For each position in the pattern, the LPS array stores the length of the longest prefix of the pattern that is also a suffix ending at that position. This array only depends on the pattern itself, so we build it once before we even touch the text.

Once we have the LPS array, the search itself becomes simple. We move through the text with one pointer and through the pattern with another. When the characters match, both pointers advance. When they do not match, we do not move the text pointer back, instead, we use the LPS array to move the pattern pointer to the right place and keep going. When the pattern pointer

reaches the end of the pattern, we have found a match and we report its starting position.

1.3 Time and Space complexity

Building the LPS array: the loop looks like it could be slow because of the inner reassignment of 'length', but each iteration either advances 'i' or decreases 'length'. Since 'length' never grows more than 'i' does, the total number of steps is bounded by $2m$. This gives us $O(m)$ time, where m is the length of the pattern.

Searching the text: the same reasoning applies here. The text pointer 'i' never goes backwards, and every step either advances 'i' or decreases 'j'. Because 'j' can only decrease as many times as it increased, the total work is bounded by $2n$. This gives us $O(n)$ time, where n is the length of the text.

Total time complexity: $O(n + m)$. This is a big improvement over the brute force approach, which would be $O(n \cdot m)$ in the worst case.

Space complexity: the only extra memory we use is the LPS array, which has one entry per character of the pattern. The two pointers take constant space. So the space complexity is $O(m)$.

For this project this is important because the transmission files are the large ones (n), while the malicious code files are comparatively small (m). Having a method that runs in linear time on the text and only needs memory proportional to the small pattern means the program stays fast even when the transmissions grow.

2. Longest Palindromic Substring

2.1 Problem Description

The second part of the project asks us to find the longest "mirrored" code inside each transmission file, which means the longest palindrome: a sequence of characters that reads the same forwards and backwards. For each transmission file we have to report where that sequence starts and where it ends, counting positions from 1.

The comparison is done at the character level, so we are looking for palindromes made of the hex characters themselves and not of their binary representation.

2.2 Algorithm

The straightforward way to solve this would be to take every position of the string, treat it as the center of a palindrome, and grow outwards while the characters on both sides keep matching. That works, but it does a lot of repeated comparisons and ends up being quadratic, which is not good enough for the large files the project mentions.

Manacher's algorithm solves the same problem in linear time. The idea behind it is that once we have found a palindrome, we already know something about the characters inside it, because a palindrome is symmetric by definition. Instead of starting from zero at every position, we reuse that knowledge.

Preparing the string

Before running the algorithm we transform the input string by inserting a separator character between every pair of characters, and also at the beginning and at the end. For example, "ABA" becomes "#A#B#A#" and "ABBA" becomes "#A#B#B#A#".

The reason for this is that palindromes come in two shapes. Odd ones like "ABA" have a character at their center, while even ones like "ABBA" have their center between two characters. Handling both cases separately would mean writing the same logic twice. After the transformation every palindrome has an odd length, so a single case covers both: an even palindrome ends up centered on one of the separators, and an odd one stays centered on its middle character.

If the original string has n characters, the transformed one has $2n + 1$.

The main loop

We build an array 'p', where 'p[i]' holds the radius of the longest palindrome centered at position 'i' of the transformed string, counting how far it reaches on each side without counting the center itself. Because of how the separators

are distributed, that radius turns out to be exactly the length of the corresponding palindrome in the original string, which is convenient since that is what we need at the end.

While we move through the string we keep track of two values, 'center' and 'right', which describe the palindrome that reaches the furthest to the right out of all the ones we have found so far. Here 'center' is its center and 'right' is its right edge. We care about the one that reaches the furthest right, and not the longest one, because the loop always moves forward: a palindrome that ended long ago tells us nothing about the position we are standing on, while one that still covers our current position does.

For each position 'i' there are two cases:

- If 'i' falls inside the palindrome described by 'center' and 'right', we can use the symmetry. The position mirrored around 'center' is $2 * \text{center} - i$, and we have already computed its value. Since the segment around 'center' is a palindrome, whatever happens around the mirror position also happens around 'i', so we can copy that value instead of comparing characters one by one. There is one limit though: the mirror is only trustworthy inside the known palindrome. If the mirrored value would take us past the right edge, we cannot trust it beyond that point, so we take the smaller of the two values.
- If 'i' falls outside the known palindrome, there is nothing to reuse and we start from zero.

After that we try to keep expanding from 'i', comparing the characters on both sides one step at a time. Since the transformed string has no padding at its ends, the loop also checks that neither index goes outside the array before comparing. Once the expansion stops, we update 'center' and 'right' if the palindrome centered at 'i' reaches further right than the previous one, and we also check whether this radius is the largest we have seen, keeping both the radius and the position where it happened.

Converting the result back

The algorithm works on the transformed string, so the last step is translating the answer back into positions of the original one. In the transformed string, a

character that sits at position k of the original string (counting from 0) ends up at position $2k + 1$, since every original character is preceded by one separator.

The palindrome we found starts at 'centerIndex - maxLen' in the transformed string, which is always a separator, so the original position is that value divided by two. Adding one converts it to the numbering the project asks for, and the ending position follows from the starting position and the length.

2.3 Time and Space complexity

Building the transformed string: we go through the original string once and add two characters for each one, so this takes $O(n)$ time.

The main loop: at first sight this looks quadratic, because there is a while loop nested inside the for loop. The important detail is that the inner loop does not run freely. Every successful iteration of the while pushes the right edge r one position further, and r never moves backwards and never goes past the end of the string. This means that across the whole execution, the inner loop can only succeed a total of n times, not n times per position. The outer loop runs n times and everything else inside it is constant work, so the whole loop is $O(n)$.

Total time complexity: $O(n)$, where n is the length of the transmission file. This is a clear improvement over the quadratic approach of expanding around every center independently, and it is what allows the program to handle large transmission files.

Space complexity: we store the transformed string, which is about twice the size of the original, and the array 'p', which has one entry per character of the transformed string. Both grow linearly with the input, so the space complexity is $O(n)$.

3. Longest Common Substring

3.1 Problem Description

The last part of the project asks us to measure how similar the two transmission files are. To do this, we need to find the longest sequence of characters that appears in both `transmission1.txt` and `transmission2.txt`, and report where that sequence starts and ends inside the first file.

It is important to notice that we are looking for a substring, not a subsequence. This means the characters have to appear one right after another, without gaps. For example, if one file contains "A1B2C3" and the other contains "XB2CY", the answer is "B2C" and not "ABC", because "ABC" is not contiguous in either file.

3.2 Algorithm

The approach we used is dynamic programming. The idea is to answer a very specific question for every pair of positions: if we stop at character `i` of the first file and at character `j` of the second file, how long is the match that ends exactly at those two positions?

We shift the indexing by one on purpose, so that position `i` represents the `i`-th character counting from 1. This saves us from checking for the edge case at the very beginning, and it also matches the output format the project asks for, which counts positions starting at 1.

Answering that question follows two simple rules:

- If the two characters are equal, the match grows by one, so we take the value computed for one character earlier in both files and add one. That earlier value is the diagonal neighbour, which belongs to the previous row.
- If the two characters are different, the run breaks completely and the value resets to zero. This is what makes this algorithm different from the longest common subsequence problem, where we would carry the previous value forward instead of resetting it.

The key observation is that each value only depends on the row immediately above it. We never look at any row older than that, and once we move past a row we never come back to it. Because of this, there is no reason to keep the whole table in memory. Two arrays of size 'm+1' are enough: one for the row we are currently filling, which we call 'curr', and one for the row we finished in the previous pass, which we call 'prev'. At the end of each pass of the outer loop, the row we just filled becomes the previous one and we move on.

While we fill each row, we keep track of the largest value we have seen so far (maxLen) and the row where it happened (endPos). That row number is exactly the ending position in the first file. Once the loops are done, the starting position is simply 'endPos - maxLen + 1', because we know how long the match is and where it ends. Recording the best result as we go is what lets us discard the older rows safely, since we never have to inspect the table again after the loops finish.

We also handle the case where the two files have nothing in common. If 'maxLen' is still zero at the end, we return zeros instead of computing a start position, since 'endPos - maxLen + 1' would otherwise give us a meaningless range.

3.3 Time and Space complexity

Time complexity: the algorithm uses two nested loops, the outer one running n times and the inner one running m times. Everything inside the inner loop is a constant amount of work: one character comparison, one assignment, and one comparison against 'maxLen'. This means we look at every one of the $n \cdot m$ combinations exactly once, which gives us $O(n \cdot m)$ time.

Copying 'curr' into 'prev' adds $O(m)$ work per pass of the outer loop, so $O(n \cdot m)$ in total. This matches the cost we were already paying to fill the rows, so the overall complexity does not change.

Space complexity: only two arrays of size 'm+1' exist at any moment, regardless of how long the first file is, so the space complexity is $O(m)$. If we always used the shorter of the two strings to size the rows, this would be $O(\min(n, m))$.

This is what keeps the memory usage practical for the large files. Storing the full table would have required one integer per pair of positions, which grows quadratically and would reach tens of gigabytes for files of a hundred thousand characters. Holding only two rows keeps the requirement in the range of kilobytes.